No 38, Jalan Riang 21, Kawasan Perindustrian Taman Gembira,
81200 Johor Bahru, Johor, Malaysia.
Tel: +607-338 3636
Email: sales@clbgroups.com

Software Design Document

Ref. No: SDD/01/260219/001

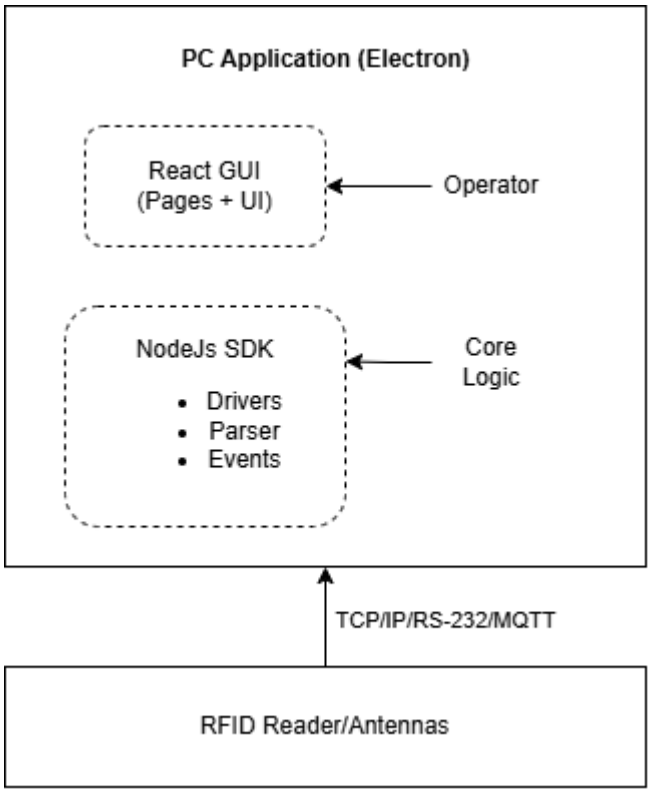
1 Purpose

This Software Design Document describes the architecture, component design, interfaces, and data flows of the Evolve SDK RFID for JJ Wine Project. It serves as a blueprint for developers, architects, and maintainers, and document design decision and trade – offs.

2 Architectural Overview

2.1 System Architecture

The system follows an event – driven architecture to ensure low latency and high availability.

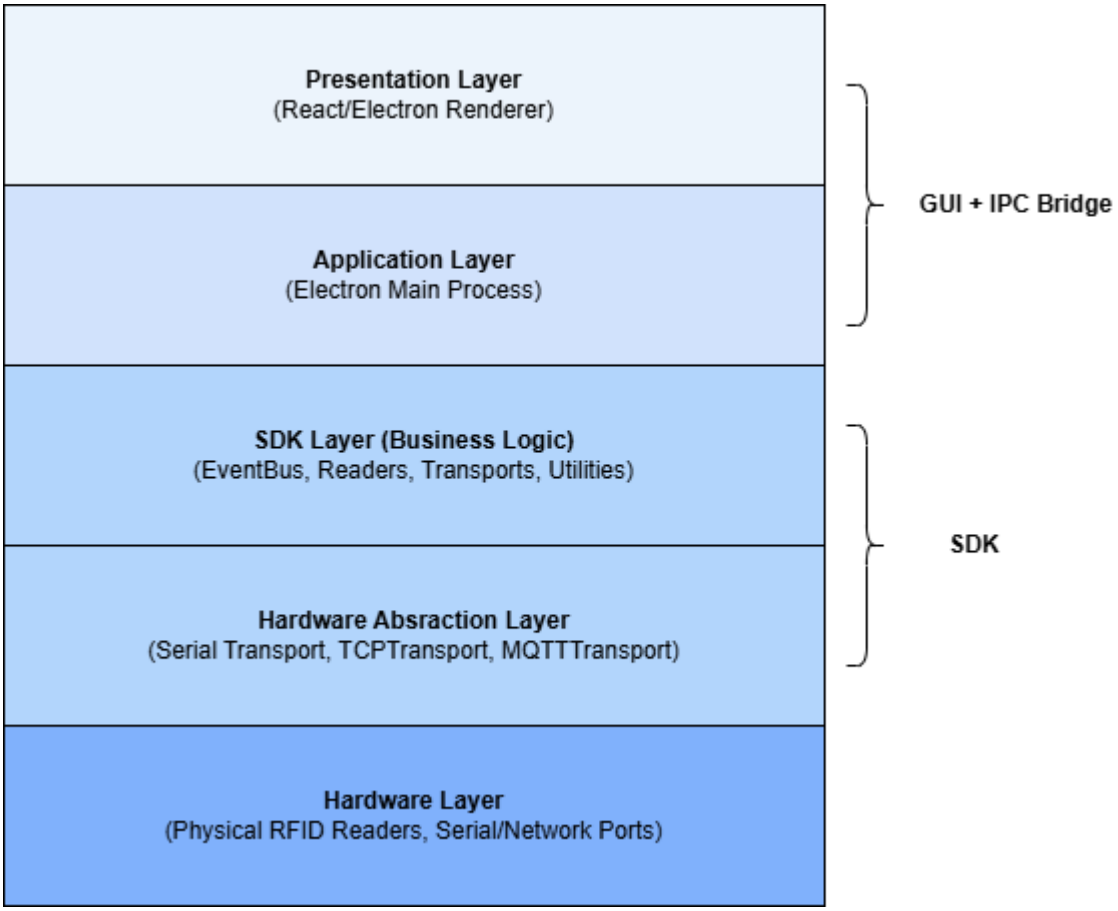


Data Flow:

1. **User** interacts with **GUI** → send commands to SDK
2. **SDK Core** validates commands, select driver → send to **reader**
3. **Reader** sends raw payload → SDK parser decodes → generates events
4. **GUI** subscribes to events → update UI (tags)
5. **Data stream** stored locally for exporting

2.2 Layered Architecture

The system utilized a **Layered Architecture** pattern to enforce separation of concerns. The critical design principle is the decoupling of the User Interface (GUI) from the Business Logic (SDK).



Key Design Principle

- **Separation of Concerns –** each layer has a single responsibility.
- **Inversion of Control –** components communicate via EventBus, not direct calls. Handles payload transformation
- **Protocol Abstraction –** all readers extend RFIDManager.
- **Transport Abstraction –** RS – 232, TCP/IP and MQTT Transports

3 Runtime Behavior

Complete end – to – end flow.

User Start → RF Scan → Tag Detection → Format → UI Update

Figure refer to *Appendix A (I).*

3.1 System Execution Overview

This section describes the complete runtime execution flow of the system during a tag reading session, beginning from the user initiation of the scanning, and ending with the display of detected tags in the graphical user interface. It is characterized by a **command – response** control flow (downwards) and **asynchronous event** data flow (upward).

The runtime behavior demonstrates:

- Command propagation from UI to hardware
- Event – driven processing within the SDK
- Hardware Abstraction through the reader drivers
- Secure IPC communication in Electron
- Reactive UI updates in React.

3.2 Scan Initialization Flow

3.2.1 User Interaction & Renderer State Update

- **Interaction:** The user initiates a scan by clicking ***"Start Read"*** button in the GUI via the *ReadControl component*
- **Local State Update:** *ReaderContext* update the scanning state to reflect that a scan session has started. This update affects UI elements (e.g., button state) but not directly interact with hardware. At this stage:
    - The system remains within the Renderer process.
    - No hardware command has been issued.

3.2.2 IPC Command Dispatch

- **Flow:** The *sdkService* module sends an IPC message to the *Electron Main* process.

reader:start – scan

- **Security Boundary:** The *preload* script validates the request. Ensuring that only authorized command reach the main process. This prevents Arbitrary Code Execution (ACE) from the UI layer

3.2.3 SDK Command Execution & Hardware Activation

1. **Management:** Upon receiving IPC event, Electron Main invokes:

   ReaderManager.startScan()

   The *ReaderManager* delegates the request to activate reader instance (e.g., UF3 – SReader).

2. **Driver Implementation:** The specific reader driver translates the high – level "*Start*" command into specific binary of ASCII protocol frame.

3. **Transport:** The transport later transmits the frame via the establish TCP/IP, Serialsocket, or MQTT.

4. **Hardware State:** The physical RFID reader enters active scanning mode and begins emitting RF signal for tag discovery.

3.3 Tag Detection and Data Processing Flow

3.3.1 RF Signal Reception

When RFID tag enters the reader's field:

1. The tags respond to the RF signal emits by the antenna.

2. The reader hardware captures the response

3. The reader transmits raw binary data through the configured communication interface.

3.3.2 Transport Layer Handling

- The transport layer receives incoming raw bytes buffers and triggers:

  onData(buffer)

- The buffer is forwarded to the reader driver for parsing

3.3.3 Payload Parsing

The reader driver performs the following critical operations on the raw buffer:

- **Frame Validation:** Verifies header and footer markers. The reader checks for specific start – of – frame (SOF) and end – of – frame (EOF) markers defined by the air interface protocol (e.g., GS1 EPC Gen2)
- **Integrity Check:** Calculates and verifies the **Checksum (CRC).** If a mismatch is detected, the packet is silently discarded to prevent data corruption.
- **Normalization:** Extract the **Electronic Product Code (EPC)** and **Received Signal Strength Indicator (RSSI),** mapping them to a standard TagData interface.

3.3.4 Event – Driven Emission

- The driver publishes the normalized ta to internal EventBus and emits:

  EventBus.emit(TAG_DETECTED, tagData)

- The EventBus provides the decoupling between reader implementation and higher – level processing logic.

3.4 Tag Aggregation and Event Forwarding

3.4.1 Reader Manager Processing

- The ReaderManager acts as an immediary subscriber. ReaderManager subcribes to *TAG_DETECTED* events and performs:
  - Deduplication (prevent rapid duplicate reads)
  - Optional counting or aggregation
  - Filtering (if configured)
- After Processing, the event is forwarded to Electron Main for IPC dispatch.

3.4.2 IPC Forwarding to GUI

- Electron Main sends:

  event:tag-detected

- This event is forwarded through the *Preload* script using a secure API exposed as:

  window.electronAPI.onTagRead(callback)

- The *Preload* layer ensures:

- Secure communication between the GUI and SDK
- No direct Node.js access in the Renderer
- Controlled API surface

3.5 UI Update Behavior

3.5.1 React Event Subscription

- The renderer subscribes to tag events via *sdkService.*

  TagContext.addTag(tagData)

- Upon receiving a tag event, the new tag is added to a list (*detectedTags* state collection list)

3.5.2 Payload Formatting

The GUI support 3 display modes:

- Hex
- Plain Text
- JSON

3.5.2.1 Display Mode State

Display format is stored in:

ReaderContext.displayFormat

Possible values:

"hex" | "text" | "json"

3.5.2.2 Formatting Responsibility

Formatting is done in the Renderer layer (React) – not inside SDK due to following reasons:

- SDK stay transport – agnostics and remains UI – independent
- GUI can switch formats dynamically without rescanning.

3.5.3 Automatic Re – render

The GUI is built with React, it follows a rule:

If the data (state) changes, the UI must change to match it.

- **Reconciliation:** React quickly compares the "old" list of tags with "new" list.
- **Update:** React identifies only the specific new tag and "injects" in to the TagList component in the GUI.

3.6 Continuous Operation Loop

After the first successful operation of tag detection, the system continues operating in loop:

1. Reader scan continuously.
2. Transport receives raw data and Reader parses buffer.
3. EventBus emits event and ReaderManager processes the event.
4. IPC forwards events and UI updates.

4 Data Flow and Interaction

4.1 Sequence Flow (Tag Read)

Refer *Appendix A (II)*

1. Reader sends raw data
2. Transport layer receives data
3. Payload processor normalizes data
4. Event dispatcher/emitter emits structured event
5. Client or GUI consumes event

For GUI Mode:

6. SDK Event → IPC Bridge → Renderer
7. Renderer display formatted payload (HEX, Plain Text, JSON)

Latency budget: ~50 ms (serial) to ~150ms (MQTT)

4.2 User Interaction Flow

Refer *Appendix A (III)*

4.3 Architecture Diagram

Refer *Appendix B(I)*

4.4 RFID Data Logic Flow

1. **Ingress:** Physical reader transmits raw binary packets via communication protocol
2. **Abstraction:** SDK validates checksums and parses packets into the ReaderManager
3. **Transport:** SDK emits a *TAG_DETECTED* events via EventEmitter
4. **Bridge:** Electron Main Process intercepts the event and forwards it via IPC to React Renderer
5. **Persistence:** Simultaneously, the event is logged into the database
6. **Egress:** User invokes the export function to query the database and generate a report.

Refer *Appendix B(II)*

4.5 RFID Data Collection & Visualization

Refer *Appendix B (III)*

5 Data Schema Design

5.1 Data Definitions

- **EPC Count:** Total number of tags read events within a time window (e.g., EPC Count = 56,351) – includes duplicates (same EPC counted multiple times).
- **Unique EPC Count:** Number of distinct EPC values detected within a time window (e.g., Unique EPC Count = 10,300) – each EPC counted once regardless of read frequency.

5.2 Time Windows

5.2.1 24 – Hour Record (Daily Session)

- Accumulates continuously during the session.
- At the end of 24 hours, final counts are persisted and session counters reset to zero.

24-hour run completed

EPC Count: 56,351

Unique EPC Count: 10,300

- Database entry:

  date: 2026-02-13

  totalEpcCount: 56351

  uniqueEpcCount: 10300

- After persistence:

  sessionTotalCount = 0

  sessionUniqueSet = empty

5.2.2 7 – Days Record

Maintains:

- Total EPC count per day (last 7 days)
- Total Unique EPC count per day (last 7 days).
- Structure:

  Day 1 → EPC: 56351, Unique: 10300

  Day 2 → EPC: 48200, Unique: 9500

  ...

  Day 7 → EPC: xxxx, Unique: xxxx

Behavior:

- Each day, a new daily record is appended.
- If more than 7 records exist, the oldest record is removed and always maintains maximum 7 entries (rolling window).

5.2.3 1 – Month Record

Maintains:

- Total EPC count per day (last 30/31 days)
- Total Unique EPC count per day (last 30/31 days)

Behavior:

- New daily record appended.
- Oldest record removed if exceeding the month length (max 30/31 entries depending on the configuration) – rolling monthly window.

5.3 Cleanup & Rotation Logic

5.3.1 Daily Cleanup (Every 24 – Hours)

At T+24 hours:

- Capture final session statistics and persist to daily record table in the database.
- Append to weekly and monthly record in database.
- Reset session counters.

5.3.2 Weekly & Monthly Cleanup

Weekly:

- If entries > 7 → remove oldest.

Monthly:

- If entries > 30/31 → remove oldest.

5.4 Data Storage Model

5.4.1 Daily Record Table

Field	Type
Date	Date
totalEPCCount	Number
uniqueEPCCount	Number

5.4.2 Weekly Record (Derived from Daily Table)

Query:

*SELECT * FROM DailyRecords*

ORDER BY date DESC

LIMIT 7

5.4.3 Monthly Record (Derived from Daily Table)

Query:

SELECT * FROM DailyRecords

ORDER BY date DESC

LIMIT 30

Note: Weekly and Monthly view are cumulative aggregations derived from daily records.

6 Mockup & Wireframe

6.1 Visily AI Mockup Design

https://app.visily.ai/projects/6ee03d98-bfd8-4ad1-adba-55bf1404c69d/boards/2467244

7 Software Development Status

7.1 Google Sheet Tracking Report

https://docs.google.com/spreadsheets/d/1ApBnYAYhVf7OmbFa4PEgXg9-kznvo0WMUfjJjbmSUN8/edit?usp=sharing

8 Sign – off

The undersigned approve the initiation of this project as defined in this document.

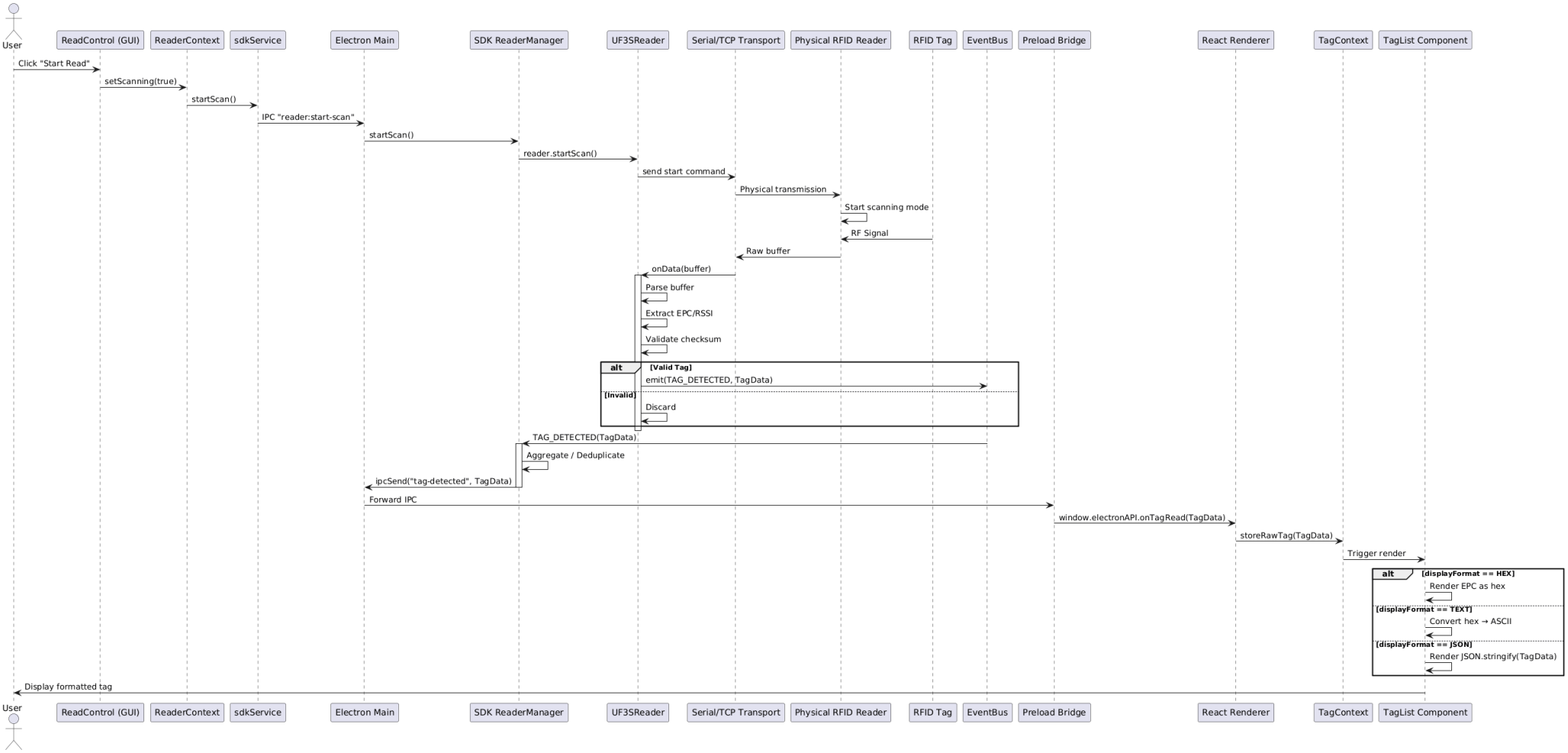
Developer/Project Manager:	**Head of Department / Manager:**
Name: Khairul Anuar	Name: Jin Ng
Date: 19th Feb 2026	Date: 26th Feb 2026

9 Appendices

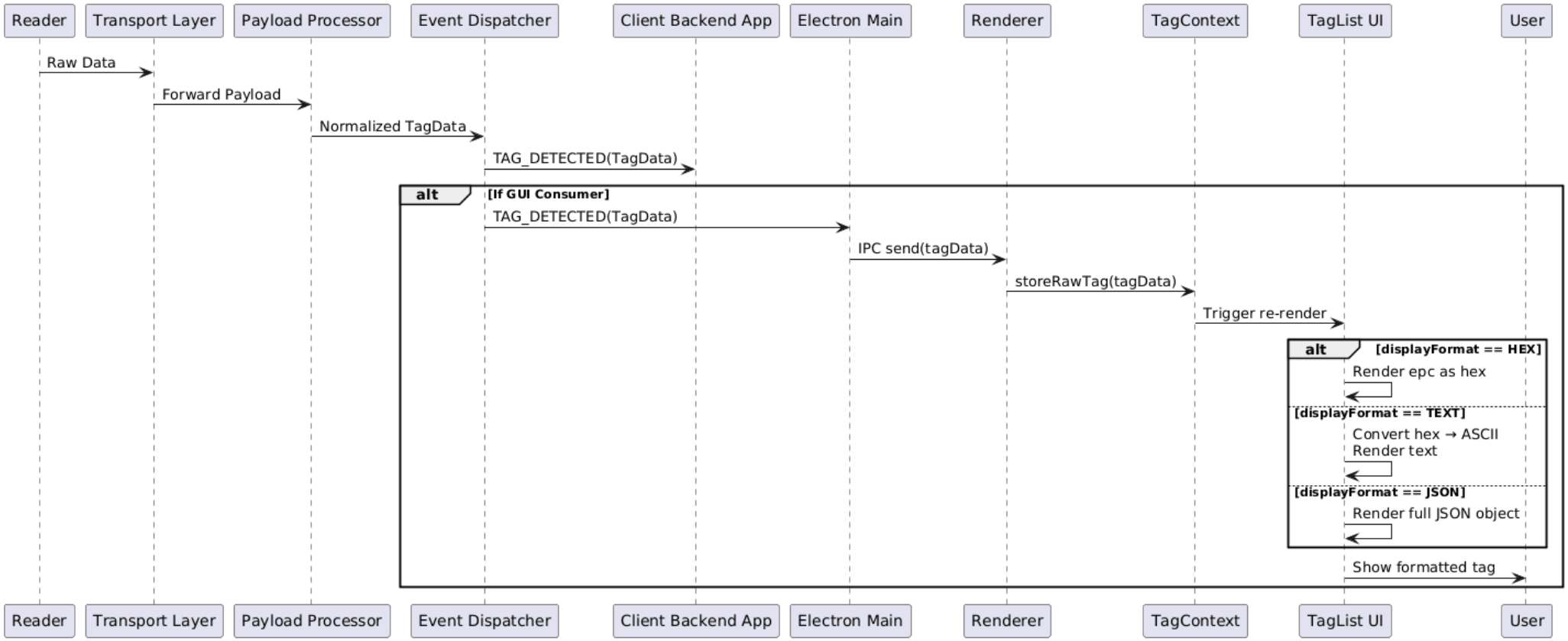
To support the document, the following appendices in included:

- ***Appendix A (I):*** Runtime Behavior
- ***Appendix A (II):*** Sequence Flow (Tag Read)
- ***Appendix A (III):*** User Interaction Flow
- ***Appendix B (I):*** Architecture Diagram
- ***Appendix B (II):*** RFID Data Logic Flow
- ***Appendix B (III):*** RFID Data Collection and Visualization Flow
- ***Appendix C:*** RFID SDK Application Lifecycle

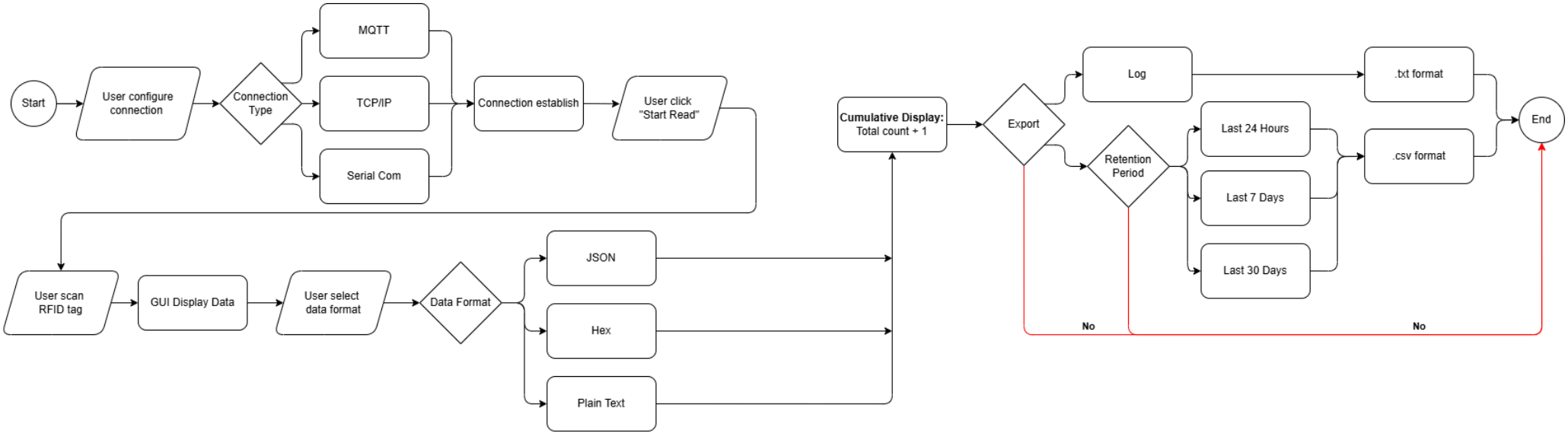
Appendix A (I): Runtime Behavior



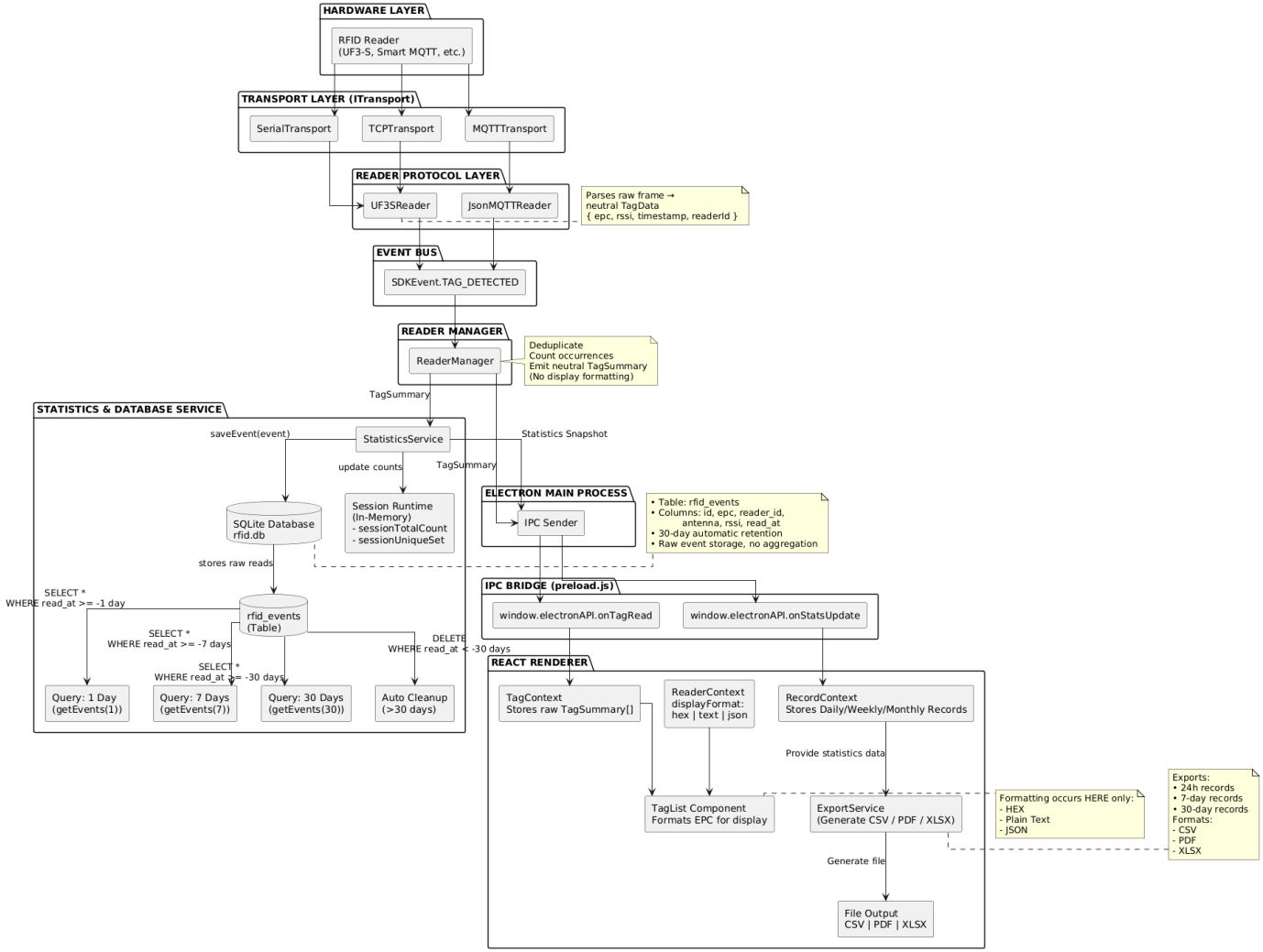
Appendix A (II): Sequence Flow (Tag Read)



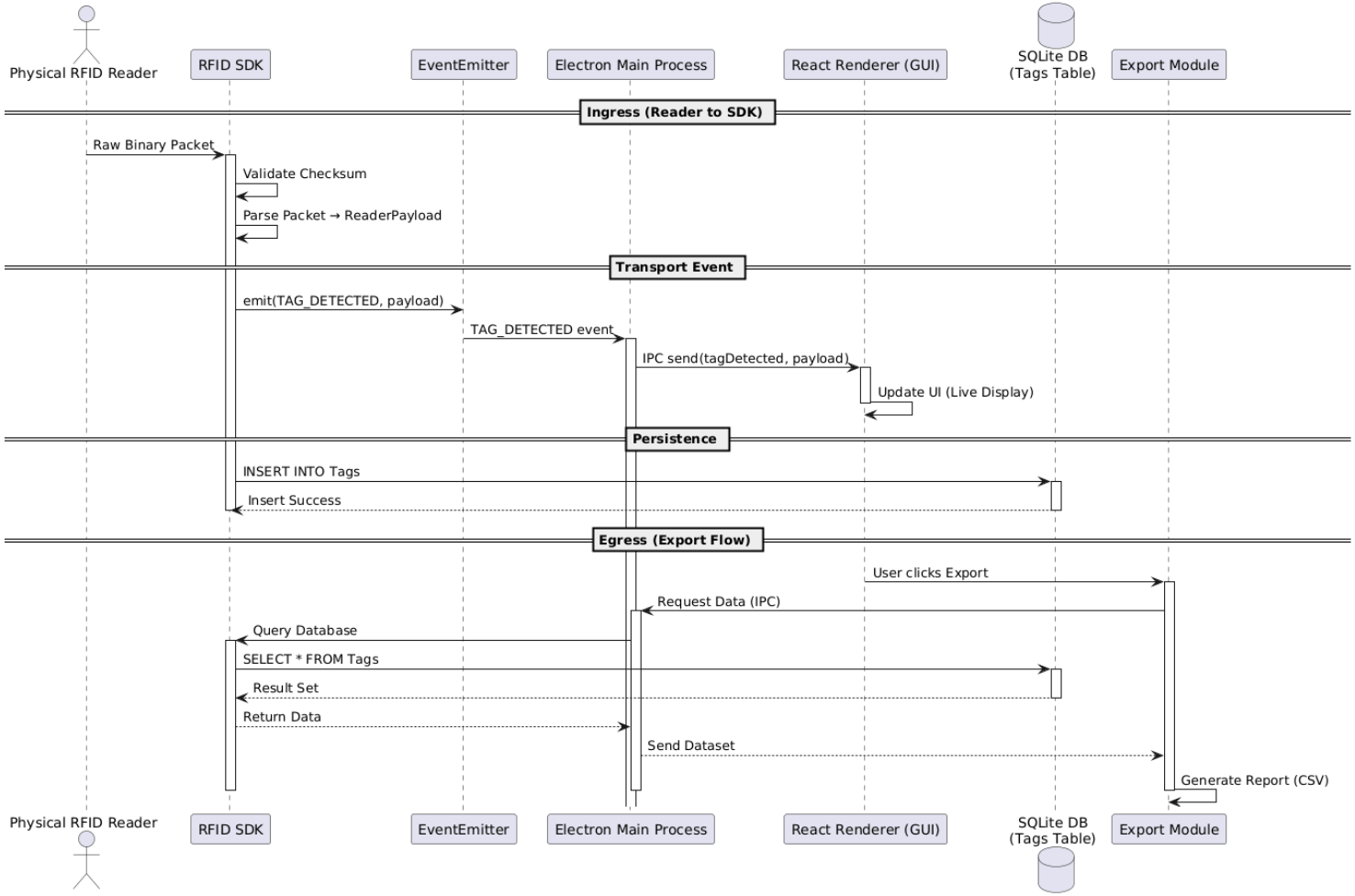
Appendix A (III): User Interaction Flow



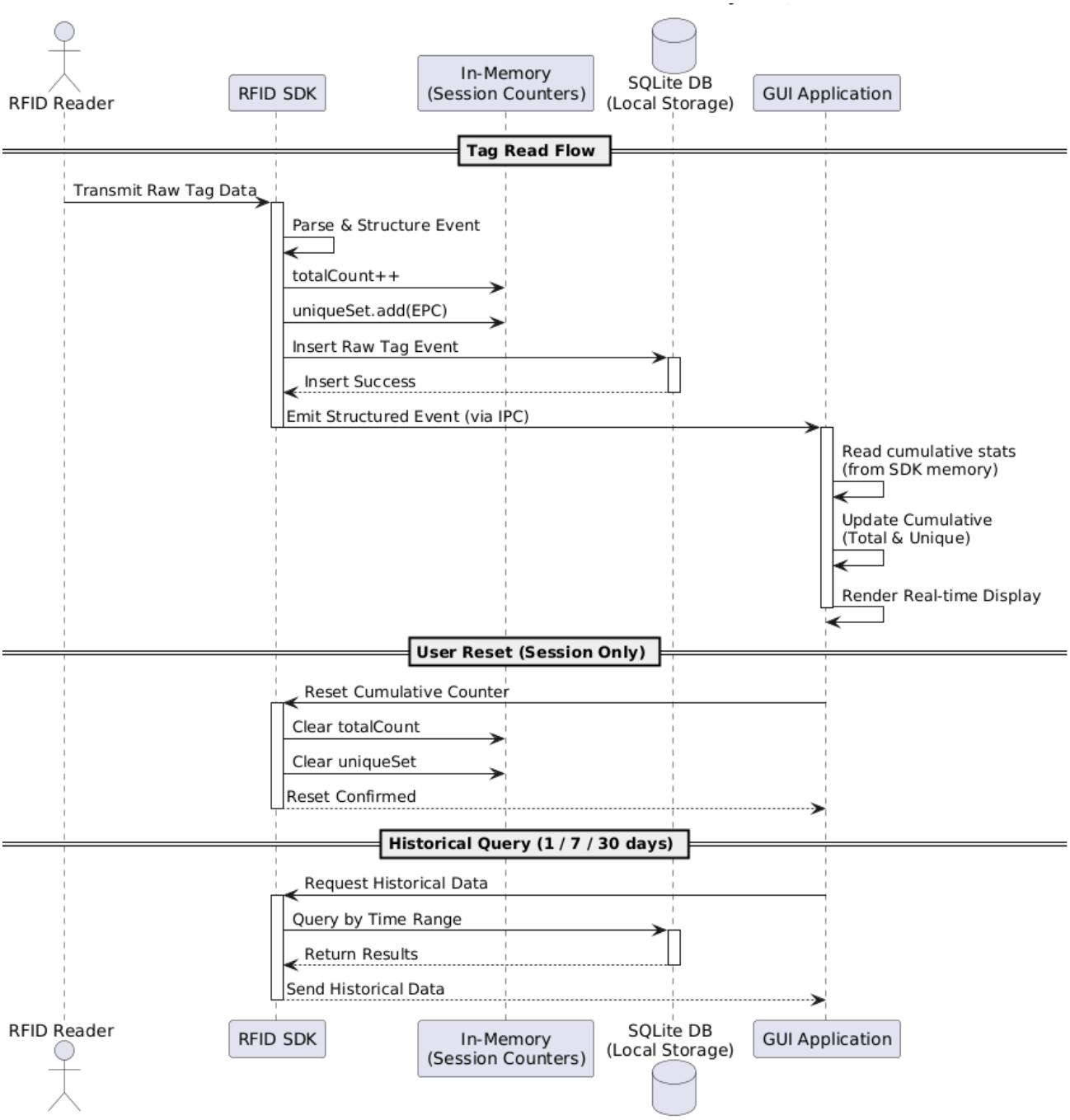
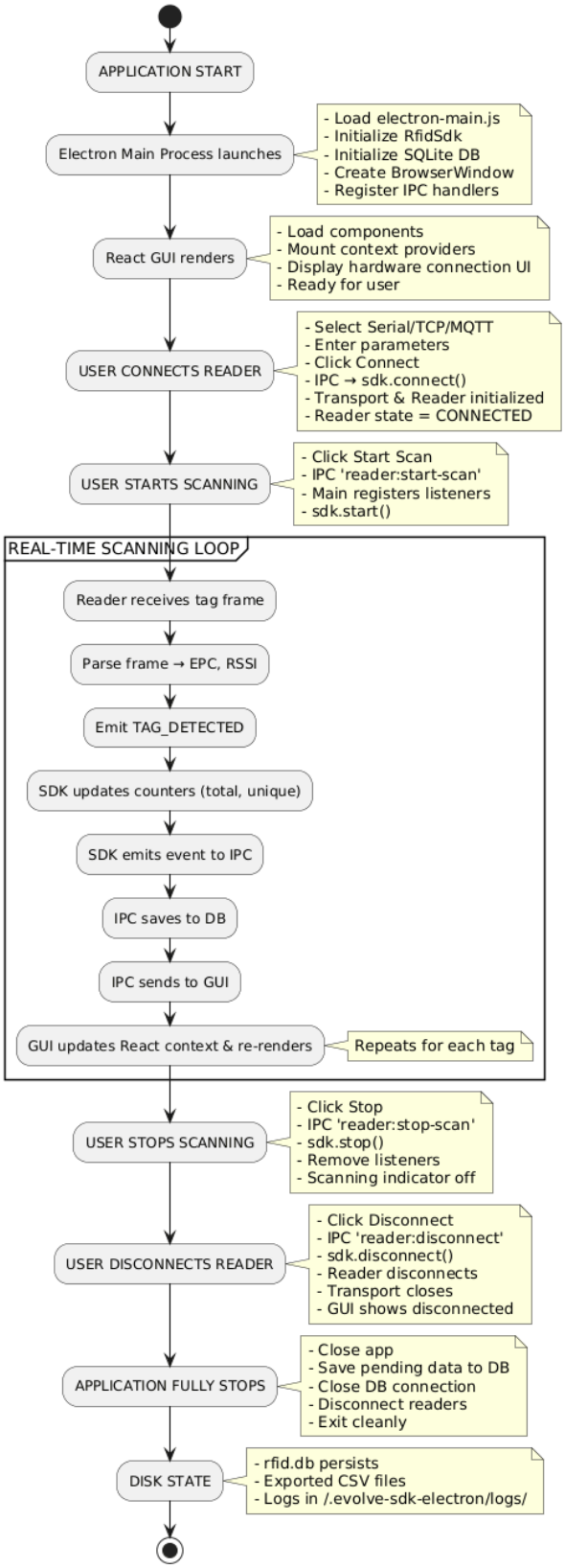
Appendix B (I): Architecture Diagram



Appendix B (II): RFID Data Logic Flow



Appendix B (III): RFID Data Collection and Visualization Flow



Appendix C: RFID SDK Application Lifecycle